

Helm Releases at Scale: Drift and Debugging



Published on March 5, 2023



Webstack
Builders

Table of Contents

Understanding Helm State	3
How Helm Tracks Releases	4
State Inconsistency Patterns	5
Drift Detection	6
Using the Helm Diff Plugin	7
Automated Drift Monitoring	8
Release Inventory Management	10
Tracking Releases Across Clusters	10
Monitoring Release Health	10
Upgrade Strategies	12
Atomic Upgrades	12
Canary and Rolling Upgrades	13
Blue-Green with Helm	13
Debugging Failed Releases	14
The Diagnosis Workflow	15
Common Failure Patterns	16
GitOps Integration	17
How GitOps Solves Drift	17
Emergency Access	19
Conclusion	20



Helm makes deploying applications to Kubernetes straightforward. Write a chart, run `helm install`, and your app materializes in the cluster. But that simplicity doesn't scale. When you're managing dozens of services across multiple clusters, you lose track of which chart versions are deployed where, which releases are drifting from their declared state, and why that upgrade failed at 2 AM.

I've spent time untangling Helm state in production clusters, and the pattern is always the same: everything works fine until it doesn't. Someone runs `kubectl edit` to hotfix a deployment during an incident. A failed upgrade leaves resources in a partially-updated state. A cleanup script accidentally deletes Helm's release secrets. Now nobody knows what's actually deployed.

Here's a scenario I've seen more than once. During an incident, an engineer tries to rollback a Helm release to a known-good state. The rollback fails. Turns out someone ran `kubectl edit` on the deployment three weeks ago to bump memory limits. Helm's stored manifest doesn't match what's in the cluster, so the three-way merge produces unexpected results. The team spends 45 minutes reconciling state instead of fixing the actual problem.

WARNING

Helm tracks its own state, but the cluster is the source of truth. When these diverge – through manual edits, partial upgrades, or failed releases – Helm's model breaks down. At scale, drift isn't **if**, it's **when**.

This article covers the operational side of Helm at scale: understanding how Helm tracks state, detecting drift before it causes incidents, maintaining a release inventory across clusters, debugging failed releases systematically, and integrating with GitOps tools for continuous reconciliation.

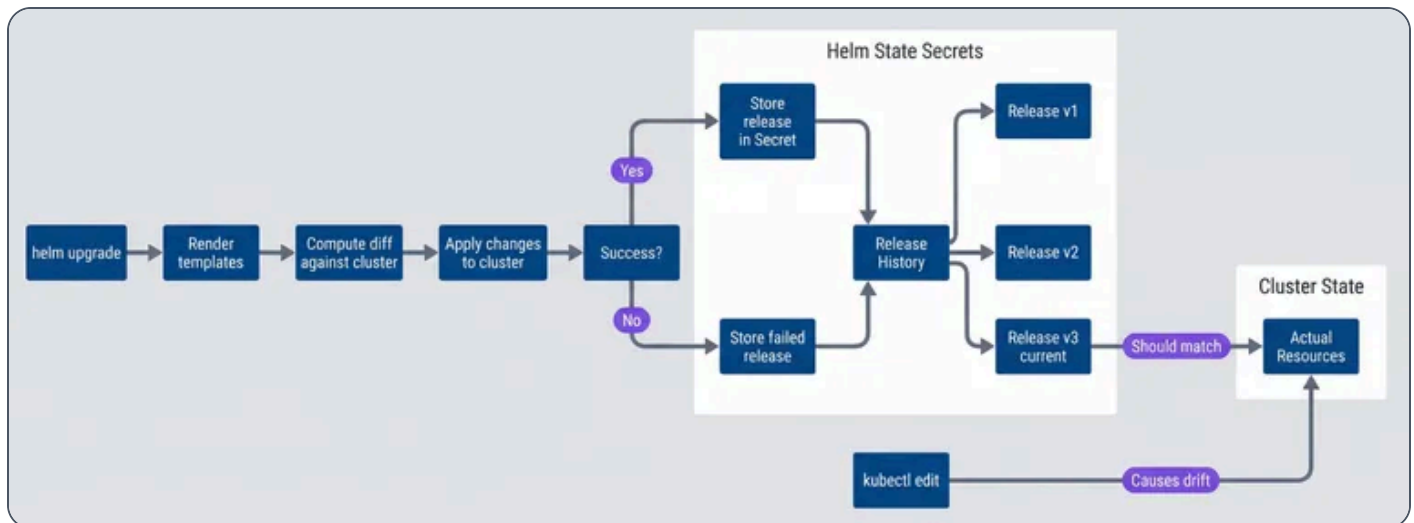
Understanding Helm State

Before you can fix drift, you need to understand how Helm tracks what it's deployed. This isn't just academic – when things break, knowing where Helm stores its state and how it compares against the cluster determines whether you spend 5 minutes or 50 minutes recovering.



How Helm Tracks Releases

When you run `helm upgrade`, Helm doesn't just push resources to the cluster and forget about them. It maintains a history of every release revision, storing the rendered manifests, values, and metadata. By default, this state lives in Kubernetes Secrets (though ConfigMaps are an option).



Helm release state and drift flow

The release secrets follow a naming convention: `sh.helm.release.v1.<release-name>.v<revision>`. Each secret contains the complete state for that revision – chart metadata, the values used, and the fully-rendered manifests. This is what enables rollbacks: Helm can reconstruct exactly what was deployed at any point in history.

helm-release-secret.yaml

```

1  # Secret name format: sh.helm.release.v1.<release-name>.v<revision>
2  apiVersion: v1
3  kind: Secret
4  metadata:
5    name: sh.helm.release.v1.my-app.v3
6    namespace: production
7    labels:
8      name: my-app
9      owner: helm
10     status: deployed

```



```
11     version: "3"
12     type: helm.sh/release.v1
13     data:
14       # Base64 + gzip encoded: chart metadata, values, rendered manifests
15       release: H4sIAAAAAAAAAA...
```

The critical thing to understand: Helm's stored manifest represents what Helm **thinks** is deployed. The cluster state represents what's **actually** deployed. When these match, operations are predictable. When they diverge, you get the fun debugging sessions.

State Inconsistency Patterns

Drift shows up in predictable patterns, and recognizing which pattern you're dealing with speeds up recovery.

Drift Type	Frequency	Severity	Recovery Difficulty
Manual kubectl edit	High	Medium	Low
Partial upgrade failure	Medium	High	Medium
Secret storage corruption	Low	Critical	High
Hook failure orphans	Medium	Medium	Medium
Three-way merge conflicts	Low	High	High

Common Helm drift patterns, their frequency, severity, and recovery difficulty.

Manual kubectl edits

The most common source. Someone runs `kubectl edit deployment` to bump resource limits during an incident, or `kubectl patch` to add an annotation. The cluster state changes, but Helm doesn't know. The next `helm upgrade` may revert those changes unexpectedly, or the three-way merge may produce surprising results.



Partial upgrade failures

These leave you in limbo. If Helm times out or hits an error mid-upgrade, some resources may be updated while others aren't. You'll see pods running different versions, ConfigMaps updated but Deployments not, and a release status of "failed" that blocks further operations.



Secret storage corruption

Less common but more painful. If cleanup scripts, namespace recreation, or manual deletion removes Helm's release secrets, the cluster still has the resources but Helm has no record of them. `helm list` shows nothing, but `helm install` fails with "already exists."



Hook failures

Creates orphaned state. Pre-upgrade or post-upgrade hooks that fail can leave partial resources. The release is marked failed, but the hook's job or pod may still exist, and main resources may or may not have been deployed depending on when the hook failed.



Three-way merge conflicts

The trickiest. When Helm computes an upgrade, it compares the old manifest, new manifest, and live cluster state. If someone modified a field in the cluster that you're also changing in the new manifest, the merge can produce unexpected results – fields deleted, values unexpectedly retained, or strategic merge patches behaving counter-intuitively.



INFO

Helm uses a three-way merge: old manifest, new manifest, live state. This usually works well, but manual changes to live state can create surprising merge results. When in doubt, use `--force` for a two-way merge (but understand the implications – it recreates resources rather than patching them).

Drift Detection

Knowing drift happens isn't enough – you need to catch it before it causes problems. Discovering drift during an incident is the worst possible time. The goal is continuous detection with alerting, so you fix divergence on your schedule rather than at 3 AM.



Using the Helm Diff Plugin

The `helm-diff` plugin is the quickest way to compare Helm's view against cluster reality. It shows what would change on an upgrade, but more importantly for drift detection, it can compare the current release against live state.

drift-check.sh

```
1  #!/bin/bash
2
3  # Install the plugin once
4  helm plugin install https://github.com/databus23/helm-diff
5
6  # Compare what Helm thinks is deployed vs what's actually in the cluster
7  helm diff revision my-release 0 --namespace production
8
9  # Preview what an upgrade would change
10 helm diff upgrade my-release ./my-chart \
11     --namespace production \
12     --values values-prod.yaml \
13     --detailed-exitcode # Exit 2 if changes, 0 if none, 1 if error
```

The `--detailed-exitcode` flag is useful for CI/CD pipelines. You can gate deployments on whether drift exists, or at minimum log a warning. I've seen teams add a pre-deployment drift check that fails the pipeline if unexpected changes exist – forcing engineers to investigate before proceeding.

For scheduled drift detection, a simple loop across releases works:

scheduled-drift-check.py

```
1  import json
2  import os
3  import subprocess
4  import urllib.request
5
6  def run(command: list[str]) -> str:
7      result = subprocess.run(command, check=False, capture_output=True, text=True)
8      return result.stdout.strip() + result.stderr.strip()
9
```



```

10 def post_slack(message: str) -> None:
11     webhook_url = os.environ.get("SLACK_WEBHOOK")
12     if not webhook_url:
13         return
14
15     payload = json.dumps({"text": message}).encode("utf-8")
16     request = urllib.request.Request(
17         webhook_url,
18         data=payload,
19         headers={"Content-Type": "application/json"},
20         method="POST",
21     )
22     urllib.request.urlopen(request, timeout=5).read()
23
24 def main() -> None:
25     releases_output = run(["helm", "list", "-n", "production", "-q"])
26     releases = [line for line in releases_output.splitlines() if line.strip()]
27
28     for release in releases:
29         drift_output = run(["helm", "diff", "revision", release, "0", "-n", "production"])
30         if drift_output:
31             print(f"Drift detected: {release} in production")
32             print(drift_output)
33             post_slack(f"Helm drift detected: {release} in production")
34
35 if __name__ == "__main__":
36     main()

```

Scheduled drift check script.

Automated Drift Monitoring

For production environments, scheduled drift checks should run as a Kubernetes CronJob. This keeps detection close to the clusters and integrates naturally with your existing monitoring.

drift-detector-cronjob.yaml

```

1  apiVersion: batch/v1
2  kind: CronJob
3  metadata:

```




```

4   name: helm-drift-detector
5   namespace: platform
6   spec:
7     schedule: "0 */4 * * *" # Every 4 hours
8     jobTemplate:
9       spec:
10        template:
11          spec:
12            serviceAccountName: helm-drift-detector
13            containers:
14              - name: detector
15                image: alpine/helm:3.14
16                command:
17                  - /bin/sh
18                  - -c
19                  - |
20                    helm plugin install https://github.com/databus23/helm-diff
21
22                    for ns in production staging; do
23                      for release in $(helm list -n $ns -q); do
24                        drift=$(helm diff revision $release 0 -n $ns 2>&1)
25                        if [[ -n "$drift" ]]; then
26                          echo "DRIFT: $release in $ns"
27                          # Push metric to Prometheus Pushgateway
28                          echo "helm_drift_detected{release=\"$release\",namespace=\"$ns\"} 1"
29
29                    | \
30                      curl --data-binary @- http://pushgateway:9091/metrics/job/helm-
31                    drift
32                      fi
33                      done
34                    done
35            restartPolicy: OnFailure

```

The ServiceAccount needs permissions to list releases and read resources across namespaces. Once metrics are flowing to Prometheus, you can alert on `helm_drift_detected == 1` and build dashboards showing drift history.



Release Inventory Management

Once you're running more than a handful of services across multiple clusters, you need visibility into what's deployed where. Questions like "which clusters are still running the old chart version?" or "do we have any failed releases?" shouldn't require logging into each cluster individually.

Tracking Releases Across Clusters

The simplest approach is periodic collection of release data from each cluster into a central store. Helm's JSON output makes this straightforward to parse:

Bash

```
1  # Get structured release data from a cluster
2  helm list --all-namespaces --output json
```

The output includes release name, namespace, chart name and version, app version, status, and last deployment timestamp. Aggregate this across clusters and you can answer questions like:

- Which releases are using chart version X vs Y?
- Which services haven't been deployed in over 30 days?
- Are there any failed releases that need attention?

For multi-cluster environments, tools like `helm-exporter` <<https://github.com/sstarcher/helm-exporter>> expose this data as Prometheus metrics automatically. This lets you build dashboards and alerts without custom collection scripts.

Monitoring Release Health

With release metadata flowing into Prometheus, you can set up useful alerts and dashboards. The key metrics to track:

helm-alerts.yaml

```
1  # Prometheus alerting rules for Helm releases
```



```

2  groups:
3  - name: helm-releases
4    rules:
5  - alert: HelmReleaseFailed
6    expr: helm_release_info{status="failed"} == 1
7    for: 5m
8    labels:
9      severity: warning
10   annotations:
11     summary: "Helm release {{ $labels.release }} in {{ $labels.namespace }} is failed"
12
13  - alert: HelmVersionInconsistency
14    expr: |
15      count by (chart) (
16        count by (chart, chart_version) (helm_release_info{status="deployed"})
17      ) > 1
18    for: 1h
19    labels:
20      severity: info
21    annotations:
22      summary: "Chart {{ $labels.chart }} has multiple versions deployed across
clusters"
23
24  - alert: HelmReleaseStale
25    expr: (time() - helm_release_last_deployed_timestamp) / 86400 > 30
26    for: 24h
27    labels:
28      severity: info
29    annotations:
30      summary: "Release {{ $labels.release }} hasn't been updated in 30+ days"

```

The version inconsistency alert is particularly useful during rollouts. If you're upgrading a chart across clusters, you can see at a glance which clusters are done and which are still pending.

Metric	Purpose	Alert Threshold
Release count by status	Track failed releases	Any failed > 1h
Version distribution	Identify inconsistencies	> 1 version per chart



Metric	Purpose	Alert Threshold
Drift detected	Cluster/Helm divergence	Any drift > 4h
Release age	Stale deployments	> 30 days without update

INFO

Export Helm release metadata as Prometheus metrics. This enables dashboards showing version consistency across clusters, alerting on failed releases, and tracking deployment frequency. The helm-exporter project provides this out of the box.

Upgrade Strategies

Not all upgrades are equal. A minor config change to a non-critical service doesn't need the same rigor as a major version bump on your payment processing system. Having a repertoire of upgrade patterns – and knowing when to use each – reduces risk and speeds up routine deployments.

Atomic Upgrades

For most production upgrades, `--atomic` should be your default. It provides transaction-like semantics: either the entire upgrade succeeds, or Helm automatically rolls back to the previous state.

Bash

```
1 helm upgrade my-release ./my-chart \
2   --namespace production \
3   --values values-prod.yaml \
4   --atomic \
5   --timeout 10m \
6   --wait
```



Without `--atomic`, a failed upgrade leaves you in an inconsistent state. Some resources are updated, some aren't, and the release is marked "failed." You're now stuck deciding whether to push forward, rollback manually, or debug. With `--atomic`, that decision is made for you – on any failure, Helm reverts everything.

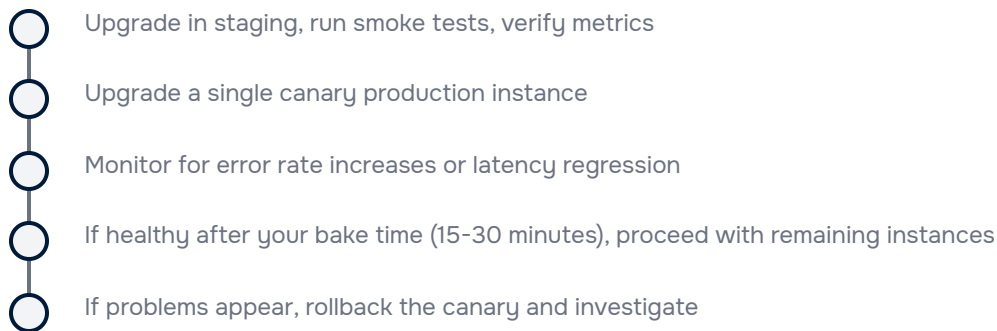
The `--timeout` flag is important. Set it long enough for your slowest pod to start, but not so long that you're waiting forever on a broken deployment. For most services, 5-10 minutes is reasonable.

Many teams use `helm upgrade --install` to handle both initial installs and subsequent upgrades with a single command. Combined with `--atomic`, this gives you idempotent deployments that work whether the release exists or not.

Canary and Rolling Upgrades

For higher-risk changes, upgrade in stages rather than all at once.

Canary upgrades deploy to a subset first – typically staging, then one production namespace or cluster – and verify before continuing. This gives you a chance to catch problems before they affect all users. The process looks like:



Rolling namespace upgrades work well for multi-tenant deployments where each namespace is isolated. Upgrade one namespace, verify, then move to the next. If something breaks, only one tenant is affected while you fix it.

Blue-Green with Helm

For zero-downtime requirements on stateless services, you can run two releases side-by-side:



Bash

```
1  #!/bin/bash
2
3  # Deploy new version as separate release
4  helm install my-app-v2 ./my-chart \
5    --namespace production \
6    --values values-prod.yaml
7
8  # Verify new release is healthy
9  kubectl rollout status deployment my-app-v2 -n production
10
11 # Switch traffic (via Ingress, Istio, or service selector update)
12 # ... update your traffic routing ...
13
14 # Once traffic is flowing to v2, remove old release
15 helm uninstall my-app-v1 --namespace production
```

Blue-green deployment with Helm.

This is more operationally complex than in-place upgrades, but it gives you instant rollback – just switch traffic back to v1 before you uninstall it.

WARNING

Always use `--atomic` for production upgrades. Without it, a failed upgrade leaves you in an inconsistent state – some resources updated, some not, release marked failed. With `--atomic`, Helm automatically rolls back on any failure.

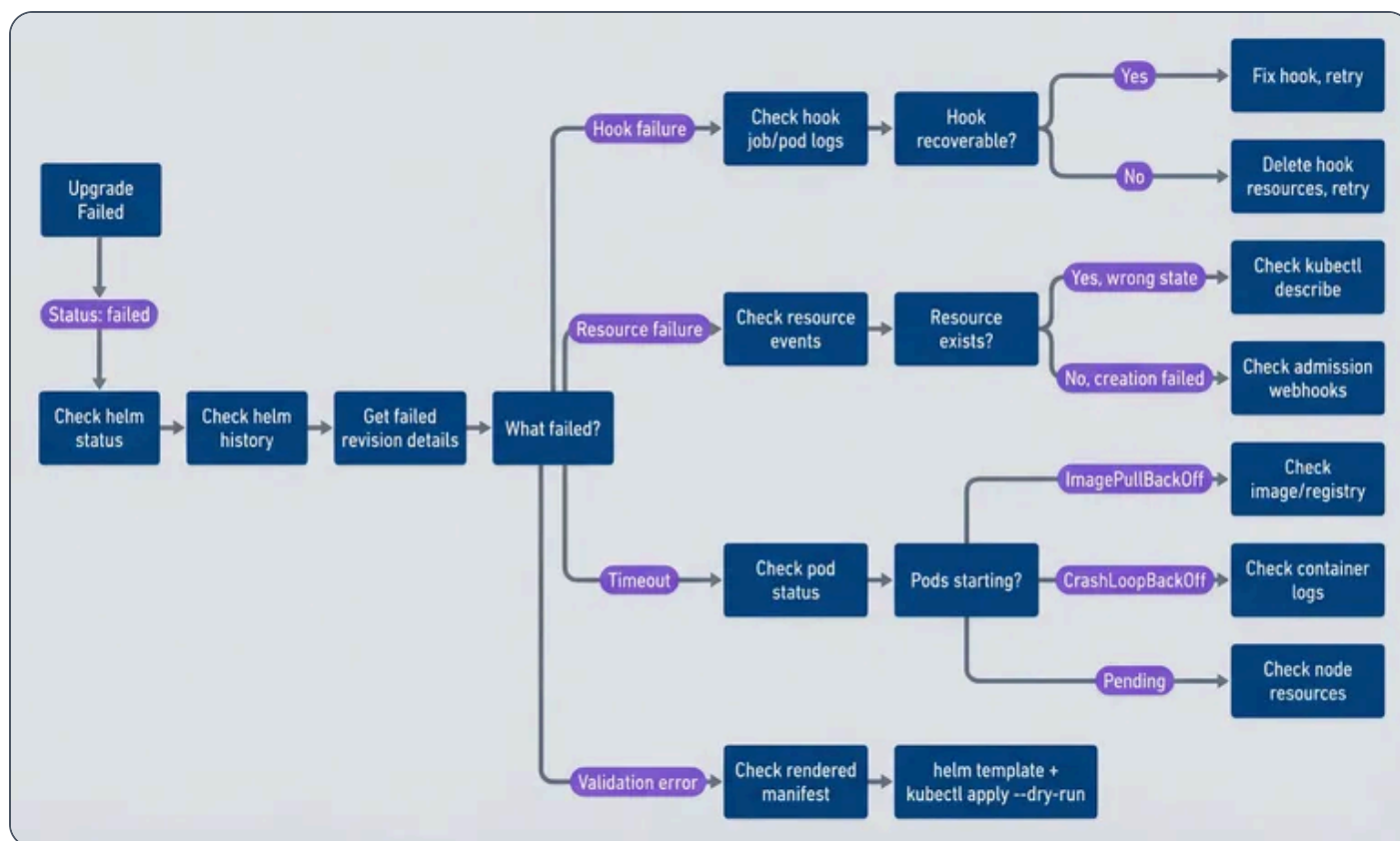
Debugging Failed Releases

When an upgrade fails, you need a systematic approach to figure out why. The error message from Helm is often just the starting point – the real cause is usually buried in Kubernetes events, pod logs, or webhook rejections.



The Diagnosis Workflow

Start with `helm status` to see the current release state, then `helm history` to see recent revisions. If the last revision shows “failed,” you need to determine what failed: a hook, a resource, a timeout, or a validation error.



Helm release failure diagnosis workflow

The key commands you’ll use repeatedly:

Bash

```

1  # See current release state and recent history
2  helm status my-release -n production
3  helm history my-release -n production
4
5  # Get the manifest and values from a specific revision
  
```



```

6  helm get manifest my-release -n production --revision 5
7  helm get values my-release -n production --revision 5
8
9  # Compare what changed between revisions
10 helm diff revision my-release 4 5 -n production
11
12 # Check Kubernetes events for the namespace
13 kubectl get events -n production --sort-by='.lastTimestamp'
14
15 # Validate rendered templates against the cluster
16 helm template my-release ./my-chart -f values.yaml | kubectl apply --dry-run=server -f -

```

Common Failure Patterns

Hook timeouts	show up as releases stuck in "pending-upgrade" or "pending-install." The release lock prevents further operations until you resolve it. Check for stuck hook jobs with `kubectl get jobs -l helm.sh/hook` and review their logs. If the hook is stuck, delete the job and either fix the underlying issue or temporarily skip hooks with `--no-hooks`.
"Resource already exists" errors	happen when a previous failed install left orphaned resources that Helm doesn't know about. You have two options: use `helm upgrade --force` to adopt the existing resources, or delete them manually and reinstall clean.
ImagePullBackOff	means Helm is waiting for pods that can't pull their images. Check the image name and tag in your values, verify your registry credentials (imagePullSecrets), and confirm network connectivity to the registry. Once fixed, run `helm upgrade --reuse-values --force`.
Admission webhook rejections	produce errors like "admission webhook denied the request." Your manifests are violating cluster policies. Use `helm template   kubectl apply --dry-run=server -f -` to see the exact rejection message, then update your values or templates to comply.
Resource quota exceeded	shows up as pods stuck in Pending with quota rejection events. Either reduce resource requests in your values, request a quota increase, or free up quota by removing unused releases.



✓ SUCCESS

When a release is stuck in “pending-upgrade,” the release lock prevents further operations. Check for stuck hooks first (`kubectl get jobs -l helm.sh/hook`). If needed, manually delete the hook job and the release secret for the pending revision, then retry.

Once you’ve stabilized your Helm operations with the practices above, there’s a more radical approach: prevent drift entirely by making Git the only way to change cluster state.

GitOps Integration

Everything I’ve described so far — drift detection, inventory tracking, safe upgrades — can be done manually or with scripts. But if you’re serious about eliminating drift as a category of problems, GitOps tools like Flux and ArgoCD change the game. They provide continuous reconciliation: the cluster state is constantly compared to Git and automatically corrected.

How GitOps Solves Drift

The core principle is simple: Git becomes the only way to change cluster state. All changes go through PRs, which means all changes are reviewed, auditable, and reversible. The GitOps controller watches the repository and applies changes automatically. If someone runs `kubectl edit` to make a manual change, the controller reverts it within minutes.

Both Flux and ArgoCD support Helm natively. Here’s what a Flux HelmRelease looks like with drift detection enabled:

flux-helmrelease.yaml

```
1  # Flux HelmRelease for GitOps-managed Helm
2  apiVersion: helm.toolkit.fluxcd.io/v2beta1
3  kind: HelmRelease
4  metadata:
5    name: my-app
6    namespace: production
```



```
7 spec:
8   interval: 5m
9   chart:
10    spec:
11     chart: my-app
12     version: "1.2.x"
13     sourceRef:
14      kind: HelmRepository
15      name: my-charts
16      namespace: flux-system
17
18 values:
19   replicaCount: 3
20   image:
21     repository: myregistry/my-app
22     tag: v2.0.0
23
24   # Automatic drift correction
25   driftDetection:
26     mode: enabled
27     ignore:
28       - paths: ["/spec/replicas"]
29       target:
30         kind: Deployment # Let HPA manage replicas
31
32   upgrade:
33     remediation:
34       retries: 3
35     remediateLastFailure: true
```

Flux HelmRelease with drift detection.

The `driftDetection.mode: enabled` setting is the key. Flux continuously compares the cluster state to what the HelmRelease declares and corrects any differences. The `ignore` block is important – some drift is intentional. If you're using a Horizontal Pod Autoscaler, you don't want Flux fighting it over replica counts.

ArgoCD achieves the same with `selfHeal: true` :



argocd-application.yaml

```
1  # ArgoCD Application for Helm chart
2  apiVersion: argoproj.io/v1alpha1
3  kind: Application
4  metadata:
5    name: my-app
6    namespace: argocd
7  spec:
8    source:
9      repoURL: https://charts.example.com
10     chart: my-app
11     targetRevision: 1.2.3
12
13   destination:
14     server: https://kubernetes.default.svc
15     namespace: production
16
17   syncPolicy:
18     automated:
19       prune: true
20       selfHeal: true # Auto-correct drift
21
22   ignoreDifferences:
23     - group: apps
24       kind: Deployment
25       jsonPointers:
26         - /spec/replicas # Managed by HPA
```

ArgoCD Application with self-healing.

Emergency Access

GitOps requires discipline. If the only way to change the cluster is through Git, what happens during an incident when you need to make a change *now*?

The answer is break-glass procedures. Create a separate ServiceAccount – something like `emergency-ops` – with elevated permissions that’s only used during incidents. Gate access behind your identity provider with MFA and time-limited credentials. When someone authenticates to this account, automatically create a ticket and send an alert to your incident channel.



The key constraint: any change made via break-glass must be reconciled back to Git within 24 hours. If you bump memory limits during an incident, open a PR the next morning to make that change permanent. If the change was a temporary workaround, revert it once the root cause is fixed. Either way, the cluster returns to a state where Git is the source of truth.

This gives you the best of both worlds: continuous reconciliation during normal operations, with an escape hatch for genuine emergencies that doesn't leave permanent drift.

INFO

GitOps tools like Flux and ArgoCD provide continuous drift detection and correction. The cluster state is continuously reconciled to match Git. Manual changes are automatically reverted. This eliminates drift as a category of problems – but requires discipline to make all changes via Git.

Conclusion

Helm's simplicity at small scale becomes operational complexity at large scale. The chart-and-upgrade model works beautifully for a handful of services, but managing dozens of releases across multiple clusters requires discipline and tooling.

The key practices that keep Helm manageable:

- **Understand Helm state**
Know where release secrets live, how three-way merge works, and what causes drift. This knowledge pays off during debugging.
- **Detect drift continuously**
Run scheduled drift checks with alerting. Don't discover drift during an incident.
- **Maintain release inventory**
Export release metadata as metrics. Track version consistency across clusters.
- **Use safe upgrade patterns**
Default to `--atomic` for production. Use canary or blue-green for high-risk changes.
- **Debug systematically**
Follow the diagnosis workflow. Check status, history, events, and logs in a consistent order.



➤ **Consider GitOps**

Flux and ArgoCD eliminate drift as a category by continuously reconciling cluster state to Git.

The tools exist. The challenge is using them consistently. Build automation that enforces good practices – drift detection CronJobs, pre-deployment checks in CI, alerting on failed releases – rather than relying on manual discipline.

✓ **SUCCESS**

The goal is confidence: confidence that you know what's deployed, confidence that cluster state matches Helm state, confidence that you can upgrade safely and rollback quickly. Build that confidence through automation, monitoring, and process – not heroics during incidents.

Copyright © 2023 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/helm-release-management-drift-detection-debugging>

